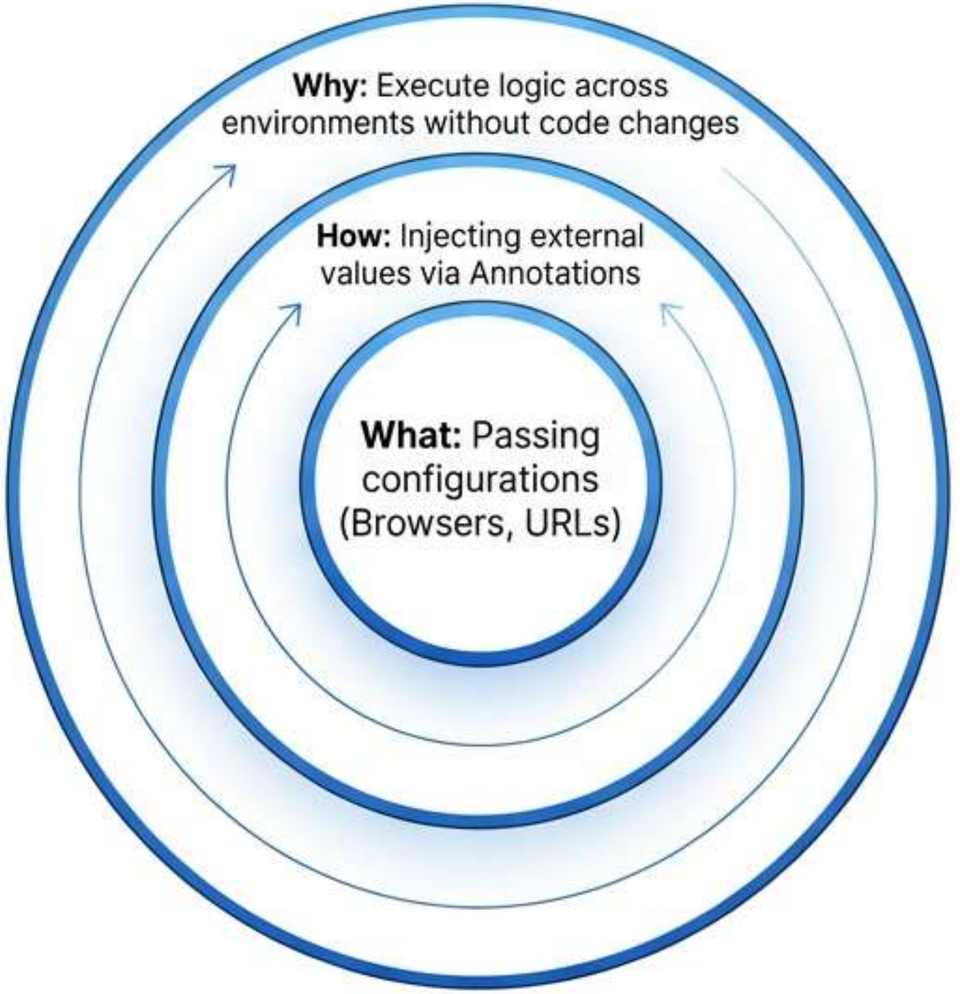
Mastering Static Parameterisation in TestNG

Strategies for Configuration Management and Data-Driven Automation

The Necessity of Data Abstraction

The Problem: Hardcoding data (URLs, credentials) creates **brittle scripts** and high maintenance overhead.

The Solution: Parameterisation acts as a **decoupling layer**, separating logic from data.



Understanding Static Parameterisation

Testleaf
Always Ahead

Definition: A method of injecting fixed configuration data defined in 'testng.xml' directly into test methods.

Static Parameterisation

- Values set in XML.
- One set of data per run.
- Ideal for Environment URLs and Browser types.

Dynamic Parameterisation

- Values from DataProviders.
- Iterative testing with multiple data sets.

Testleaf
Always Ahead

The Mechanics of Implementation

testng.xml

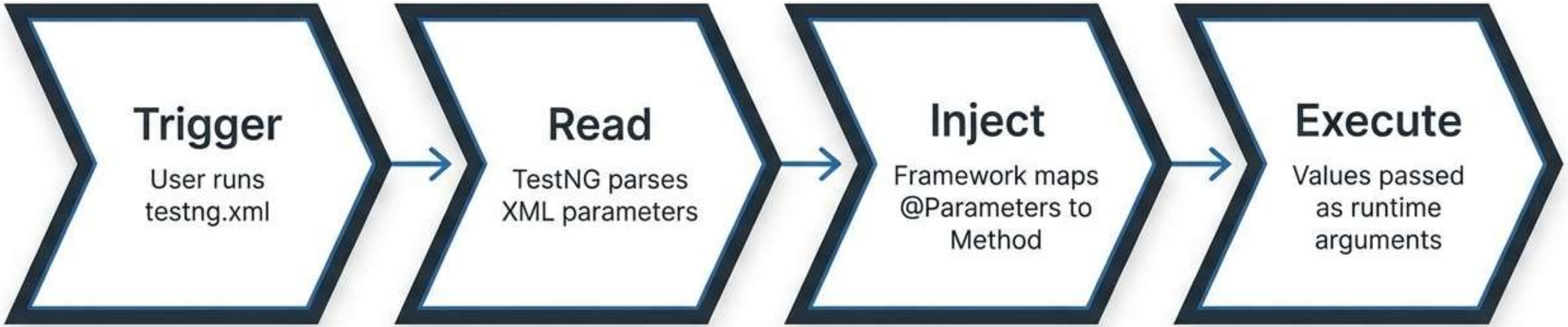
```
<parameter name="username"
      value="demosalesManager"/>
```

Java Class

```
@Parameters({"username"})
public void login(String uName) {
}
```

The Annotation Rule: The @Parameters value must match the XML name attribute exactly.

Execution Architecture



Data flows from the Configuration File into the Automation Engine.

Practical Application: Environment Management

```java
public class BaseClass {
    public ChromeDriver driver;

    @Parameters({"url", "username", "password"})
    @BeforeMethod
    public void preCondition(String url, String uName, String pwd) {
        WebDriverManager.chromedriver().setup();
        driver = new ChromeDriver();
        driver.get(url);
        driver.findElement(By.id("username")).sendKeys(uName);
        driver.findElement(By.id("password")).sendKeys(pwd);
    }
}
```

Defines required inputs

Receives values from XML

Testleaf
Always Ahead

Challenge 1: Implementation Pitfalls

The ‘Parameter Not Found’ Exception

XML Configuration

```
<parameter name="user"
  value="admin"/>
```

Java Test Annotation

```
@Parameters({"username"})
```

- **Cause:** Key in testng.xml does not strictly match the string in @Parameters.
- **Consequence:** TestNG throws a fatal exception.
- **Solution:** Always copy-paste names to verify exact mapping.

Core Competencies Recap

Decoupling

Separates environment config from test logic.

Flexibility

Enables cross-environment testing via XML only.

Syntax

Strict mapping of XML tags to Java annotations.

Scope

Global (Suite) vs Local (Test) management.

Executive Summary

We have transitioned from brittle, hardcoded scripts to a flexible, configuration-driven architecture. Mastery of 'testng.xml' is now as critical as the Java logic itself.